Low Level C++ Theory & Applications

Amlal El Mahrouss

Ne

I’m Amlal! MTS @ Ne.app, nice to meet you all!

Email: amlal@ne-app.eu
Blog: https://amlal.nekernel.org
Ne.app: https://ne-app.eu

Purpose.

→ In this talk we're going to layout techniques and decisions to take in order to successfully develop a low-level system in modern C++.

The standards involved:

→ C++17 and later.

Ne

I: First Principles.

Amlal El Mahrouss

Observation: What we will cover.

→ Why C++ over C? (In N% percent of times)

→ How C++ evolved (in a good way).

→ What to learn from C++ in your codebase.

II: Common RAII.

Amlal El Mahrouss

Observation: construct, use, destruct:

When using non-freestanding C++ RAII is mostly a ‘non-brainer’:

→ std::unique_ptr.
→ std::shared_ptr.

However, one may judge to roll its own container when deemed necessary, such as Open C++ Libraries, ‘tracked_ptr’, or Boost’s custom expansions of shared pointers operations such as delete_ptr.

Observation: construct, use, destruct:

Per ANT resource handling in HAL.dll: It's architecture makes use of the RAII pattern to manage resources that are deemed to be eligible for such operations.

For example, file descriptors that are used by a routine are automatically freed after being run. Or HAL entries being cleared outside of memory for memory reasons.

Such operations are: Load, Unload, Call, Add, Remove.

Observation: construct, use, destruct:

Another example: A resource allocator for a graphics engine.

This is a non-trivial operation, where the complexity of the algorithm may be exponential here. Thus per RAII, we will need to perform caching and implement a pool allocation algorithm.

This helps for:

- Linearity of the search.
- Searching time per algorithm complexity.

III: Several RAII Patterns.

Amlal El Mahrouss

IV: Applications and Examples of RAII.

Amlal El Mahrouss

V: Final Remarks.

Amlal El Mahrouss

VI: Conclusion.

Amlal El Mahrouss